

Contents

□ Tree DFS Patterns — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	12
SECTION 16 — Optimization Techniques	12
SECTION 17 — Multiple Language Templates	13
SECTION 18 — Dry Runs	14
SECTION 19 — Advanced Concepts	14
SECTION 20 — Senior Engineer Insights	15
SECTION 21 — Cheat Sheet	15
SECTION 22 — Revision Notes (5-Minute Review)	15
SECTION 23 — Practice Roadmap	15
SECTION 24 — Memory Tricks	16
SECTION 25 — Final Summary	16

□ **Tree DFS Patterns — Complete Interview Handbook**

Pattern #13 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Binary Tree section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Tree DFS (Depth-First Search) explores a tree by going as deep as possible along each branch before backtracking, using recursion (or an explicit stack). The three canonical orders — **preorder** (root, left, right), **inorder** (left, root, right), and **postorder** (left, right, root) — determine *when* the current node is processed relative to its children, which fundamentally shapes what kinds of problems each order naturally solves.

1.2 Why Was This Pattern Invented?

Trees are recursively-defined structures (a tree is a root plus subtrees, which are themselves trees) — DFS is the traversal that directly mirrors this recursive definition, allowing a problem on the whole tree to be expressed as a combination of the same problem solved on its subtrees. This recursive decomposition is the natural, minimal-code way to process hierarchical data.

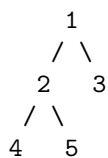
1.3 Real Intuition Behind The Pattern

Imagine exploring a **family tree genealogy chart** by picking a person, then fully investigating one of their children’s entire lineage before moving to the next child — you go as deep as possible down one branch before backtracking to explore a sibling branch, which is exactly DFS.

1.4 Mental Model

“What do I need to know about my children before I can answer for myself?” (postorder — bottom-up aggregation, like height/diameter/sum computations) versus “What do I do with myself before telling my children what I know?” (preorder — top-down propagation, like passing down a running path sum) versus “What natural order does visiting nodes produce?” (inorder — sorted order for BSTs).

1.5 Visual Explanation



Preorder (root, left, right): 1, 2, 4, 5, 3

Inorder (left, root, right): 4, 2, 5, 1, 3

Postorder (left, right, root): 4, 5, 2, 3, 1

1.6 Simple Analogy

Tree DFS is like **reading a company’s org chart by fully investigating one manager’s entire reporting chain before moving to the next manager at the same level** — you commit fully to one branch’s depth before ever backtracking to a sibling.

1.7 When Should I Immediately Think About Using This Pattern?

- Any problem on a **binary tree or n-ary tree** requiring visiting every node.
- “Height,” “depth,” “diameter,” “sum of paths,” “balanced tree check” — all bottom-up (postorder) aggregations.
- “Serialize/deserialize,” “path from root to leaf” — top-down (preorder) propagation.
- “Validate BST,” “kth smallest in BST” — inorder traversal (naturally sorted for BSTs).

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"height," "depth," "diameter"	Postorder (bottom-up) DFS
"path sum," "root-to-leaf"	Preorder (top-down) DFS, often with backtracking
"kth smallest," "validate BST"	Inorder DFS
"serialize/deserialize"	Preorder DFS with null markers
"balanced," "same tree," "subtree of another tree"	Postorder DFS (needs children's results first)
"lowest common ancestor"	Postorder DFS with bottom-up result propagation

2.2 Hidden Hints

If the answer for a node **depends on results from its children**, it's postorder. If the answer for a node **depends on information passed down from its ancestors**, it's preorder. If the problem cares about **left-to-right sorted order** (common in BSTs), it's inorder.

2.3 Interview Clues

Interviewer draws a tree and asks about aggregating a property "up" the tree (sum, height, count) — a bottom-up (postorder) signal; or asks about tracking something "down" the tree (path so far, running max) — a top-down (preorder) signal.

2.4 Common Trick Words

"Lowest common ancestor," "diameter," "maximum path sum" (these often require returning *two* things from each recursive call: the value used by the parent, and a possibly-different global answer tracked via a side variable).

2.5 What Interviewers Expect

Correct choice of traversal order matching the problem's dependency direction, correct handling of the recursive base case (null node), and — for "maximum path sum"-style problems — correctly distinguishing between "what this subtree contributes upward" versus "the best answer seen anywhere so far" (often requiring a class-level/closure variable alongside the return value).

2.6 When NOT To Use This Pattern

- You need **level-by-level** processing (e.g., "print each level," "find the rightmost node per level") — that's Tree BFS (Pattern #14), not DFS.
- The tree is extremely deep (imbalanced, essentially a linked list) and recursion risks **stack overflow** — consider an iterative DFS with an explicit stack, or reconsider whether a different traversal order avoids the deep dependency chain.
- You need to process nodes in **insertion/priority order** unrelated to tree structure — that's a different data structure (heap) or traversal entirely.

SECTION 3 — Decision Framework

Does the problem need to visit every node in the tree?

Yes

Does the answer for a node depend on its CHILDREN's results?

Yes → USE POSTORDER DFS (compute children first, then combine at the node)

No

Does the answer for a node depend on information from its ANCESTORS (passed down)?

Yes → USE PREORDER DFS (process node first, pass state down to children)

No

Does the problem care about LEFT-TO-RIGHT SORTED ORDER (typically for BSTs)?

Yes → USE INORDER DFS

No

Does the problem need LEVEL-BY-LEVEL processing?

Yes → USE BFS (Pattern #14) instead, not DFS

Why: The three DFS orders differ precisely in *when* the current node's own processing happens relative to its children — matching this timing to the problem's actual data dependency (bottom-up vs. top-down vs. sorted-order) is the single most important design decision in any tree DFS problem.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: A tree's recursive definition ($\text{Tree} = \text{Empty} \mid \text{Node}(\text{value}, \text{LeftSubtree}, \text{RightSubtree})$) directly maps to a recursive function definition: $f(\text{Empty}) = \text{base case}$; $f(\text{Node}(v, L, R)) = \text{combine}(v, f(L), f(R))$. This is **structural induction** — if f is correct on empty trees (base case) and correctly combines results assuming f is correct on L and R (inductive step), then f is correct on all trees by induction on tree structure/size.

Intuitive: Since every node has strictly smaller subtrees as children (a tree of size n has children of size $< n$), recursion is guaranteed to terminate, and correctness follows directly from mathematical induction on the size of the (sub)tree.

Correctness Proof (general form): *Base case:* $f(\text{null})$ = some well-defined base value, trivially correct (e.g., height of an empty tree is 0, sum is 0). *Inductive hypothesis:* assume f correctly computes the desired property for any tree with fewer than n nodes. *Inductive step:* for a tree with n nodes rooted at node, both node.left and node.right have fewer than n nodes, so by the inductive hypothesis $f(\text{node.left})$ and $f(\text{node.right})$ are correct; combining them correctly with node.value (per the problem's specific combine logic) yields the correct result for the full tree. **QED by structural induction.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. Define the **base case**: what should the function return/do for a null node?
2. Define the **recursive calls**: what do you need from the left and right subtrees?
3. Define the **combine step**: how do you use the current node's value plus the children's results to produce the answer for this node?
4. Decide the **order** (pre/in/post) based on whether the combine step needs children's results first (postorder), needs to pass info down first (preorder), or needs sorted left-to-right visiting (inorder).

5.2 General Template — Postorder (Bottom-Up)

```
function dfs(node):
    if node is null:
        return baseCaseValue

    leftResult = dfs(node.left)
    rightResult = dfs(node.right)

    return combine(node.value, leftResult, rightResult)
```

5.3 General Template — Preorder (Top-Down)

```
function dfs(node, stateFromParent):
    if node is null:
        return

    newState = updateState(stateFromParent, node.value)
    processIfNeeded(node, newState)

    dfs(node.left, newState)
    dfs(node.right, newState)
```

5.4 Inorder Template (BST-specific)

```
function inorder(node, result):
    if node is null:
        return
    inorder(node.left, result)
    result.append(node.value)
    inorder(node.right, result)
```

5.5 Interview Thinking Process

1. "I'll identify whether this node's answer needs its children's results (postorder) or needs information from ancestors (preorder), or needs sorted order (inorder)."
2. "I'll define the base case for a null node explicitly — this is often where subtle bugs occur."
3. "If I need both 'what this subtree contributes to its parent' and 'the best answer anywhere in the tree,' I'll track the global best via a side variable/closure, separate from the return value."
4. "I'll verify correctness via structural induction: base case + correct combination assuming children are already correct."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$	$O(h)$ recursion stack, $h = \text{height}$ ($O(n)$ for skewed tree, $O(\log n)$ for balanced)	Every node visited exactly once; stack depth bounded by tree height
Average Case	$O(n)$	$O(\log n)$ for balanced trees	Balanced trees keep recursion depth logarithmic
Best Case	$O(n)$ (must visit all nodes for most problems)	$O(\log n)$ to $O(1)$	Even simple existence checks may need full traversal in the worst case
Amortized	$O(n)$ total across the single traversal	$O(h)$	No repeated work — each node processed once

Skewed tree risk: a completely unbalanced tree (essentially a linked list) has height $O(n)$, meaning $O(n)$ recursion stack depth — a real stack-overflow risk for very large, unbalanced trees in production code.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty tree	<code>root = null</code>	Base case must handle this gracefully (return 0, null, empty list, etc. per problem semantics)
Single node	[5]	Both children are null — base case applies directly to leaf's children
Completely skewed tree (linked-list-like)	Every node has only a left child	$O(n)$ recursion depth — risk of stack overflow for very large n
Duplicate values in tree	Multiple nodes with the same value	Clarify whether the problem treats duplicate values specially (e.g., BST validation with strict vs non-strict inequality)
Negative values (path sum problems)	Node values can be negative	“Maximum path sum” must handle the case where including a negative subtree contribution would hurt — clamp contribution to 0 if negative in some variants
Path must go through a specific node type (leaf only)	“root-to-leaf path sum”	Must explicitly check <code>node.left == null</code> and <code>node.right == null</code> for “leaf,” not just <code>node == null</code>

Edge Case	Example	Handling
Tree with only one child at some nodes	Node has left but not right child	Base case correctly returns default value for the missing child without crashing

Common mistakes: conflating “leaf node” (no children) with “null node” (doesn’t exist) — leading to off-by-one errors in path-length/height calculations; forgetting to handle negative-value subtrees correctly in max-path-sum problems (including a net-negative branch when it shouldn’t be included).

SECTION 8 — Pros & Cons

Advantages: Directly mirrors the tree’s recursive structure, leading to concise, provably-correct-by-induction code; naturally handles arbitrary tree shapes without special-casing. **Disadvantages:** Recursive implementations risk stack overflow on deeply skewed trees; some problems (like “maximum path sum”) require careful separation of “return value” from “global answer,” which can be conceptually tricky. **Trade-offs:** Recursive DFS (concise, risk of stack overflow) vs. Iterative DFS with explicit stack (more verbose, avoids recursion-depth limits) — prefer iterative for production code handling potentially very large/unbalanced trees. **Limitations:** Not suitable for level-by-level processing (needs BFS instead); doesn’t inherently handle very wide/shallow trees any differently than narrow/deep ones in terms of code structure, but performance characteristics (stack depth) differ significantly. **Inefficient when:** N/A for its exact use case (visiting every node) — $O(n)$ is optimal; the “inefficiency” risk is really about stack depth/space, not time.

SECTION 9 — Real World Applications

Domain	Application
Compilers	Abstract Syntax Tree (AST) traversal for code generation, optimization passes, and type checking — all classic DFS applications
Databases	Query execution plan tree evaluation (e.g., evaluating a SQL query’s operator tree bottom-up)
File Systems	Directory tree traversal (calculating total folder size bottom-up is a classic postorder DFS)
XML/JSON Parsers	DOM tree traversal and manipulation
Version Control (Git)	Traversing commit trees/directory trees for diffing and merging
Game Development	Scene graph traversal (rendering nested game objects, computing transformed positions)
Organizational Systems	Org chart traversal for reporting hierarchies, permission inheritance computation
Machine Learning	Decision tree traversal for inference (following a path from root to a leaf prediction)
Search Engines	Trie/tree-based autocomplete traversal (related to Pattern #16)

Domain	Application
Networking	Spanning tree traversal in network topology discovery protocols

SECTION 10 — Interview Strategy

How seniors answer: They immediately state which traversal order the problem needs and *why* (based on data dependency direction), explicitly separate “value returned to parent” from “global best answer” when needed (e.g., max path sum), and reason about correctness via structural induction rather than just “trust me, it works.”

How juniors answer: They often write DFS code by pattern-matching a memorized template without articulating why that specific order was chosen, and they frequently conflate the “local contribution” and “global answer” concepts in problems like Diameter of Binary Tree or Maximum Path Sum, leading to subtly wrong logic.

Typical follow-ups: “Can you do this iteratively instead of recursively?” “What’s the space complexity in terms of tree height, not just node count?” “How would this change for an n-ary tree instead of a binary tree?” “What if the tree is extremely unbalanced — any stack overflow risk?”

Optimization questions: “Can you avoid recomputing height/depth repeatedly across multiple calls?” (memoize or combine into a single postorder pass returning multiple values, e.g., height AND balanced-check together, rather than two separate traversals).

SECTION 11 — Pattern Variations

Variation	Description	Example
Preorder DFS	Root before children — top-down propagation	Path Sum, Serialize/Deserialize Binary Tree
Inorder DFS	Left, root, right — sorted order for BSTs	Validate BST, Kth Smallest Element in a BST
Postorder DFS	Children before root — bottom-up aggregation	Maximum Depth, Diameter of Binary Tree, Balanced Binary Tree
Dual-Return DFS	Return both “local contribution” and track “global best” separately	Binary Tree Maximum Path Sum, Diameter of Binary Tree
DFS with Path Tracking	Carry a path/list down through recursion, record on leaves	Path Sum II, Binary Tree Paths
DFS with Memoization	Cache results for repeated subtree structures	House Robber III
Morris Traversal (O(1) space)	Threaded-tree technique avoiding recursion/stack entirely	Inorder/Preorder Traversal without extra space

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Tree BFS	Level-by-level, queue-based, not depth-first	Need per-level processing, shortest path in unweighted tree/graph sense
Backtracking	Also DFS-based, but explicitly tracks/undoes a mutable “choice” state for enumeration	Generating all paths/combinations, not just computing one aggregate value
Graph DFS/BFS	Generalizes tree DFS to structures with cycles, requiring visited-tracking	Data isn’t a tree (has cycles or isn’t strictly hierarchical)
Dynamic Programming on Trees	Tree DFS with memoization when overlapping subtree computations exist	Repeated subtree evaluation needed (rare in simple trees, common in DAG-like structures)

Comparison Table

Aspect	Tree DFS	Tree BFS	Graph DFS
Traversal order	Depth-first (one branch fully before next)	Level-by-level	Depth-first, but needs visited-tracking (cycles possible)
Space	$O(h)$ — tree height	$O(w)$ — tree width (max level size)	$O(V)$ — visited set
Best for	Aggregation (height, sum, path), sorted order (BST)	Level-order, shortest path (unweighted)	Connectivity, cycle detection in general graphs

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Simple traversal, single aggregate value	Maximum Depth of Binary Tree, Same Tree, Invert Binary Tree
Medium	Path tracking, BST-specific properties	Path Sum II, Validate Binary Search Tree, Kth Smallest Element in a BST
Hard	Dual-return (local + global), complex combination logic	Binary Tree Maximum Path Sum, Diameter of Binary Tree (subtle version), Serialize and Deserialize Binary Tree
Very Hard	Multi-tree combination, advanced state tracking	Binary Tree Cameras, House Robber III (with memoization), Recover Binary Search Tree

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Maximum Depth of Binary Tree	Easy	Amazon, Meta, Microsoft, Google	Direct postorder height computation	Foundational postorder mechanics
2	Same Tree	Easy	Amazon, Meta	Simultaneous preorder comparison	Dual-tree traversal
3	Invert Binary Tree	Easy	Amazon, Meta, Google	Preorder/postorder swap	Basic tree mutation
4	Symmetric Tree	Easy	Amazon, Meta, Microsoft	Mirrored dual-tree traversal	Mirror comparison logic
5	Path Sum	Easy	Amazon, Meta	Preorder with running sum	Top-down state passing
6	Path Sum II	Medium	Amazon, Meta	Preorder with path tracking + backtracking	Path collection with backtracking
7	Balanced Binary Tree	Easy	Amazon, Meta, Microsoft	Postorder with early-exit optimization	Combined height + validity check
8	Diameter of Binary Tree	Easy/Medium	Amazon, Meta, Google, Microsoft	Postorder with dual local/global tracking	Dual-return DFS mastery
9	Binary Tree Maximum Path Sum	Hard	Amazon, Meta, Google, Microsoft	Postorder with dual local/global tracking + negative handling	Advanced dual-return DFS
10	Validate Binary Search Tree	Medium	Amazon, Meta, Google, Microsoft	Inorder traversal or range-passing preorder	BST-specific traversal choice
11	Kth Smallest Element in a BST	Medium	Amazon, Meta, Google	Inorder traversal with early termination	Sorted-order DFS application
12	Lowest Common Ancestor of a Binary Tree	Medium	Amazon, Meta, Google, Microsoft	Postorder with bottom-up result propagation	Advanced postorder combination
13	Lowest Common Ancestor of a BST	Easy/Medium	Amazon, Meta	BST-property-guided traversal	Exploiting BST structure

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
14	Serialize and Deserialize Binary Tree	Hard	Amazon, Meta, Google, Microsoft	Preorder with null markers	Constructive preorder DFS
15	Binary Tree Paths	Easy	Amazon, Google	Preorder with path string building	Path construction via DFS
16	Sum Root to Leaf Numbers	Medium	Amazon, Meta	Preorder with running numeric state	Numeric state propagation
17	Flatten Binary Tree to Linked List	Medium	Amazon, Meta, Microsoft	Postorder-based tree restructuring	In-place tree mutation
18	House Robber III	Medium	Amazon, Meta, Google	Postorder DFS with two-state return (rob/not-rob)	Tree DP via DFS
19	Binary Tree Cameras	Hard	Amazon, Google, Meta	Postorder DFS with greedy state propagation	Advanced tree greedy + DFS
20	Count Good Nodes in Binary Tree	Medium	Amazon, Meta	Preorder with running max state	Top-down state tracking
21	Construct Binary Tree from Preorder and Inorder Traversal	Medium	Amazon, Meta, Microsoft, Google	Recursive construction using traversal properties	Traversal-based construction
22	Recover Binary Search Tree	Hard	Amazon, Google	Inorder traversal anomaly detection	Advanced inorder application
23	Vertical Order Traversal of a Binary Tree	Hard	Amazon, Meta, Google	DFS with coordinate tracking + sorting	Coordinate-augmented DFS
24	Subtree of Another Tree	Easy	Amazon, Meta	Postorder comparison at every node	Nested traversal comparison
25	Count Univalued Subtrees	Medium	Amazon, Google	Postorder with validity propagation	Bottom-up validity aggregation
26	Delete Nodes And Return Forest	Medium	Amazon, Google	Postorder with deletion + forest construction	Advanced tree restructuring

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
27	Distribute Coins in Binary Tree	Medium	Amazon, Google	Postorder with flow/balance propagation	Advanced postorder flow calculation
28	Smallest String Starting From Leaf	Medium	Amazon, Google	Preorder/postorder with string comparison	Path-based string construction
29	Maximum Width of Binary Tree	Medium	Amazon, Meta	DFS with coordinate/index tracking (contrast with BFS approach)	Coordinate-based DFS variant
30	All Nodes Distance K in Binary Tree	Medium	Amazon, Meta, Google	DFS to build parent pointers + BFS combination	Cross-pattern (DFS + BFS combination)

SECTION 15 — Common Mistakes

1. Conflating “leaf node” with “null node” — causing off-by-one errors in height/depth/path-length calculations. *Fix:* explicitly check `node.left == null && node.right == null` for “leaf,” distinct from the null-node base case.
2. In “maximum path sum”-style problems, returning the same value both “up to the parent” and as “the final answer,” without recognizing these can differ (a path through a node using both children can’t be extended further up). *Fix:* track the global answer via a side variable while returning only the single-branch contribution to the parent.
3. Forgetting to clamp negative subtree contributions to zero in max-path-sum problems (including a net-negative branch only hurts the sum). *Fix:* `contribution = max(0, childResult)` where applicable.
4. Choosing the wrong traversal order (e.g., preorder when the problem actually needs children’s results first). *Fix:* always ask “does this node’s answer depend on its children, or on its ancestors?” before coding.
5. Not considering stack-overflow risk for very deep/skewed trees when recursion is used in production-quality code. *Fix:* mention the iterative-with-explicit-stack alternative when discussing trade-offs.

Why people fail: the three traversal orders look superficially similar (just reordering three lines of code), but choosing the wrong one for a given problem’s data dependency produces code that’s syntactically valid but semantically incorrect — and problems like Maximum Path Sum specifically test whether a candidate can correctly separate “local contribution” from “global answer,” a distinction many candidates blur under pressure.

SECTION 16 — Optimization Techniques

- **Time:** Combine multiple traversals into one where possible (e.g., compute height AND check balance in a single postorder pass, returning a sentinel value like -1 to signal “already unbal-

anced,” rather than two separate $O(n)$ and $O(n^2)$ traversals).

- **Space:** Consider Morris Traversal for $O(1)$ space inorder/preorder traversal (using temporary tree-threading) when recursion/stack space is a genuine constraint.
 - **Readability:** Clearly comment which traversal order is used and why; separate “local return value” and “global answer tracking” into clearly named variables in dual-return problems.
 - **Interview performance:** Explicitly state the chosen traversal order and its justification before coding — this is the single highest-signal habit for tree DFS problems.
-

SECTION 17 — Multiple Language Templates

Java

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

JavaScript

```
function maxDepth(root) {
    if (!root) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

PHP

```
function maxDepth($root): int {
    if ($root === null) return 0;
    return 1 + max(maxDepth($root->left), maxDepth($root->right));
}
```

Python

```
def max_depth(root):
    if root is None:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

Go

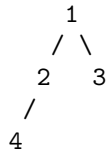
```
func maxDepth(root *TreeNode) int {
    if root == nil {
        return 0
    }
    left := maxDepth(root.Left)
    right := maxDepth(root.Right)
    if left > right {
        return left + 1
    }
    return right + 1
}
```

C++

```
int maxDepth(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(maxDepth(root->left), maxDepth(root->right));  
}
```

SECTION 18 — Dry Runs

Small Input



maxDepth(1):

```
maxDepth(1) = 1 + max(maxDepth(2), maxDepth(3))  
maxDepth(2) = 1 + max(maxDepth(4), maxDepth(null)) = 1 + max(1, 0) = 2  
maxDepth(4) = 1 + max(maxDepth(null), maxDepth(null)) = 1 + max(0, 0) = 1  
maxDepth(3) = 1 + max(0, 0) = 1  
maxDepth(1) = 1 + max(2, 1) = 3
```

Large Input (Conceptual)

For a balanced tree with 10^6 nodes (height ≈ 20), the recursion depth is only ~ 20 , well within typical stack limits, and the traversal visits each of the 10^6 nodes exactly once — $O(n)$ time, $O(\log n)$ space.

Corner Case

root = null: maxDepth(null) = 0 immediately via the base case, correctly representing an empty tree's depth. Single node [5]: maxDepth(5) = 1 + max(maxDepth(null), maxDepth(null)) = 1 + 0 = 1, correctly representing a single-node tree's depth.

SECTION 19 — Advanced Concepts

- **Dual-return DFS (Diameter, Max Path Sum):** the key insight is that a node's contribution to its *parent* (a single downward path through one child) is different from the *best answer possible through this node* (potentially using both children) — track the former as the return value, the latter via a side variable updated at every node.
- **Range-passing BST validation:** rather than inorder-traversing into a list and checking sortedness ($O(n)$ space), pass down a valid (min, max) range to each recursive call, narrowing it appropriately for each child — achieving the same correctness with less overhead and no separate list.
- **Morris Traversal:** achieves $O(1)$ space traversal by temporarily threading the tree (making a node's inorder predecessor point back to it), traversing, then un-threading — an advanced technique for space-constrained tree traversal without recursion or an explicit stack.
- **Tree DP (House Robber III, Binary Tree Cameras):** postorder DFS returning a **tuple/pair of states** (e.g., “best if this node is included” vs “best if excluded”) generalizes

tree traversal into full dynamic programming over tree structure — an important bridge between Tree DFS and DP patterns.

SECTION 20 — Senior Engineer Insights

Staff engineers view Tree DFS as the concrete embodiment of **structural/mathematical induction applied to code** — every correct tree recursion is, at its core, a proof by induction on tree size, and articulating this connection demonstrates CS fundamentals depth. They also recognize that many “hard” tree problems (Maximum Path Sum, Binary Tree Cameras, House Robber III) are fundamentally about correctly identifying **what information must flow bottom-up versus what must be tracked as a separate global state** — a skill that generalizes directly to distributed systems problems like aggregating metrics up a service dependency tree, or computing rollup permissions in a nested organizational hierarchy.

SECTION 21 — Cheat Sheet

PATTERN: Tree DFS

RECOGNIZE: any binary/n-ary tree problem needing to visit every node

TRAVERSAL CHOICE:

Postorder (children first) → aggregation: height, sum, diameter, balanced check

Preorder (node first) → top-down state: path sum, serialize, count-good-nodes

Inorder (left, node, right) → sorted order: BST validation, kth smallest

TEMPLATE (postorder):

```
function dfs(node):  
    if node is null: return baseCase  
    left = dfs(node.left); right = dfs(node.right)  
    return combine(node.value, left, right)
```

COMPLEXITY: $O(n)$ time, $O(h)$ space (h = tree height)

KEY PROOF: structural induction - base case (null) + correct combination assuming children are correct

WATCH FOR: leaf vs null confusion, local-vs-global answer separation, negative-value clamping, stack over

DOESN'T APPLY WHEN: need level-by-level processing (use BFS), data has cycles (use Graph DFS with visited)

SECTION 22 — Revision Notes (5-Minute Review)

- Three orders: preorder (top-down state), inorder (sorted, BST), postorder (bottom-up aggregation).
 - Correctness proof = structural induction: base case (null) + correct combine assuming children are correct.
 - Dual-return pattern: track “contribution to parent” separately from “global best answer” (Diameter, Max Path Sum).
 - Clamp negative contributions to 0 in max-path-sum style problems.
 - Leaf $\neq$ null — always distinguish explicitly.
 - Recursion depth = tree height; skewed trees risk stack overflow — mention iterative alternative.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic traversal orders and aggregation	Maximum Depth of Binary Tree (104), Same Tree (100), Invert Binary Tree (226)
Intermediate	Path tracking, BST properties	Path Sum II (113), Validate Binary Search Tree (98), Kth Smallest Element in a BST (230)
Advanced	Dual-return, LCA, construction	Diameter of Binary Tree (543), Lowest Common Ancestor of a Binary Tree (236), Construct Binary Tree from Preorder and Inorder Traversal (105)
Expert	Complex combination, tree DP	Binary Tree Maximum Path Sum (124), House Robber III (337), Binary Tree Cameras (968)

SECTION 24 — Memory Tricks

- **Mnemonic:** “**Pre**-visit before kids, **In**-order between kids, **Post**-visit after kids” — matches the position of “root” (node processing) in the traversal name itself.
 - **Visualization:** A **family tree investigator** who fully explores one child’s entire lineage before moving to the next sibling.
 - **Recognition shortcut:** “Depends on children” → postorder. “Depends on ancestors/path so far” → preorder. “Sorted order / BST” → inorder.
-

SECTION 25 — Final Summary

Tree DFS directly mirrors a tree’s recursive definition, making correctness provable via structural induction: a correct base case for null nodes, combined with correctly combining already-correct results from smaller subtrees, guarantees correctness for the whole tree. The single most important thing to remember forever: **choose your traversal order by asking whether the current node’s answer depends on its children (postorder), on information from its ancestors (preorder), or on sorted left-to-right order (inorder) — and in “dual-return” problems like Diameter or Maximum Path Sum, always separate what a subtree returns to its parent from the best global answer, tracked independently.**